

Homework Bio Informatica 8 Maggio

Nome: Samuele Cognome: Bergamin Matricola: 844861

1 Costruzione dei vettori **W**, **Last** ed **F**

Per comprendere la proprietà *rank preserving*, è preliminarmente necessario spiegare come vengono costruiti i vettori di base della struttura DBG compressa.

I singoli k-mer del grafo di de Bruijn vengono inseriti nel vettore **Node** seguendo un ordine co-lessicografico (o lessicografico invertito). L'ordinamento viene effettuato leggendo ogni singolo k-mer dalla lettera più a destra verso quella più a sinistra. Ad esempio, es. $TC\$ > ACA$, leggendo i k-mer al contrario abbiamo infatti $\$CT > ACA$.

Una volta ottenuto il vettore **Node**, viene costruito il vettore **W**. Esso contiene le etichette degli archi uscenti di ogni singolo k-mer del grafo, in modo tale che nell' i -esima entry di **W** sia contenuta l'etichetta dell'arco uscente dal k-mer posizionato all' i -esima entry del vettore **Node**. In caso di molteplici archi uscenti dallo stesso nodo, il k-mer ad esso associato occuperà più righe contigue, garantendo che gli archi uscenti dallo stesso k-mer siano adiacenti in **W**.

Il vettore **Last** serve a delimitare i nodi e gestire i molteplici archi uscenti. Le celle vengono riempite con il valore 0 in corrispondenza degli archi intermedi di un nodo, mentre assumono il valore 1 esclusivamente in corrispondenza dell'ultimo arco uscente di un dato k-mer.

Infine, il vettore **F** memorizza gli indici da cui iniziano i k-mer raggruppati in base alla loro ultima lettera. Nell'esempio riportato nell'homework, $F[T] = 14$ indica che alla posizione 14 inizia il blocco dei k-mer che terminano con la lettera T.

2 Ricerca dell' i -esima lettera **T** (senza segno meno)

Per individuare in quale entry del vettore **Last** si trova la destinazione dell' i -esima lettera **T** letta nel vettore **W**, si effettuano le seguenti operazioni.

Innanzitutto, si calcola il rango del carattere letto:

$$R = \text{rank}_T(W, i)$$

Questa funzione restituisce il numero di occorrenze del carattere **T** presenti tra la posizione 0 e la posizione i del vettore **W**.

Per via della proprietà *rank preserving*, è garantito che l' i -esima occorrenza di **T** in **W** punti esattamente all' i -esimo k-mer all'interno del blocco individuato da **F[T]**. Poiché ogni k-mer è delimitato univocamente da un valore 1 nel vettore **Last**, per trovare la posizione esatta (la riga) in cui termina l' i -esimo k-mer, è necessario individuare l' i -esimo 1 all'interno del vettore **Last**, iniziando il conteggio a partire dall'indice **F[T]**.

Matematicamente, la riga di destinazione si calcola estraendo la posizione tramite la funzione **select**:

$$\text{Destinazione} = \text{select}_1(\text{Last}, \text{rank}_1(\text{Last}, \text{F}[\text{T}] - 1) + R)$$

Non è sufficiente spostarsi di $R - 1$ posizioni assolute, bensì è necessario scorrere $R - 1$ celle contenenti il valore 1 nel vettore **Last**. Le posizioni con valore 0 non identificano nuovi nodi, ma solo archi addizionali dello stesso k-mer, e vanno pertanto ignorate nel conteggio dei nodi di destinazione.

3 Giustificazione del segno meno

Mentre il vettore **Last** (con i suoi 0) permette di gestire la presenza di molteplici archi *uscenti* da un singolo nodo, il segno meno in **W** è stato introdotto per gestire le collisioni derivanti da molteplici archi *entranti* nello stesso nodo (in-degree > 1).

Nel caso in cui vi siano molteplici archi $\{e_1, e_2, \dots, e_n\}$ che confluiscono nel medesimo nodo di destinazione, solo il primo arco (secondo l'ordinamento co-lessicografico) viene registrato con il carattere standard in **W**. A tutti i successivi archi entranti viene apposto un segno meno (es. **T-**). I simboli marcati col segno meno vengono di fatto ignorati dalla funzione **rank**, impedendo che archi ridondanti sfasino il conteggio dei nodi successivi e garantendo il mantenimento strutturale della proprietà *rank preserving*.

4 Implementazione di Outdegree(v)

Per calcolare il numero di archi uscenti (**outdegree**) da un nodo il cui ultimo arco si trova alla riga v , si procede come segue:

1. Identificazione lessicografica del nodo: Si determina il "numero" del nodo attuale contando le occorrenze di 1 fino alla posizione v .

$$R = \text{rank}_1(\text{Last}, v)$$

Poiché i valori 1 indicano l'ultimo arco di ciascun nodo, sono presenti tanti 1 quanti sono i nodi unici incontrati.

2. Individuazione del nodo precedente: Viene eseguita una query per recuperare l'indice in cui terminava il nodo precedente, utilizzando la funzione **select** per trovare l'($R - 1$)-esimo 1:

$$\text{nodo_prec} = \text{select}_1(\text{Last}, R - 1)$$

Questa operazione restituisce la riga dell'ultimo arco uscente del nodo immediatamente antecedente a v .

3. Calcolo finale: Il numero di archi uscenti si ottiene sottraendo l'indice del nodo precedente dall'indice attuale:

$$\text{Outdegree}(v) = v - \text{nodo_prec}$$

Caso limite: Nel caso in cui il nodo in analisi sia il primo in assoluto del grafo ($R = 1$), non essendoci alcun nodo precedente, il calcolo si semplifica in:

$$\text{Outdegree}(v) = v + 1$$

5 Implementazione del LF-mapping

L'operazione di *Forward mapping* o LF-mapping ($j = \text{Forward}[i]$) permette di ricavare la riga di destinazione j seguendo l'arco posto alla riga i . I passaggi logici sono i seguenti:

1. Identificazione del carattere: Sia $c = W[i]$ il carattere dell'arco uscente alla posizione i (assumendo valori non negativi).
2. Calcolo del rango in W: Si effettua il conteggio delle occorrenze di c dalla posizione 0 alla posizione i inclusa:

$$\text{Rango} = \text{rank}_c(W, i)$$

3. Individuazione del blocco in F: Si individua l'indice di partenza del blocco associato al carattere c nel vettore F:

$$\text{Inizio_Blocco} = F[c]$$

4. Conteggio dei nodi precedenti: Si determina il numero totale di nodi presenti nel grafo *prima* che inizi il blocco c , contando i valori 1 nel vettore **Last**:

$$\text{Nodi_Precedenti} = \text{rank}_1(\text{Last}, \text{Inizio_Blocco} - 1)$$

5. Calcolo del nodo bersaglio: Sommando i nodi precedenti al rango calcolato al punto 2, si ottiene l'identificativo assoluto del nodo di destinazione:

$$\text{Target} = \text{Nodi_Precedenti} + \text{Rango}$$

6. Mappatura sulla riga di destinazione (j): Infine, poiché ogni 1 in **Last** chiude uno specifico nodo, è sufficiente effettuare una query **select** per recuperare l'indice esatto:

$$j = \text{select}_1(\text{Last}, \text{Target})$$

L'equazione complessiva risulta quindi:

$$j = \text{select}_1(\text{Last}, \text{rank}_1(\text{Last}, F[W[i]] - 1) + \text{rank}_{W[i]}(W, i))$$

6 Esercizio sui Suffix Tree

6.1 Ricerca delle Maximal Repeat in T

Una stringa rappresenta una *maximal repeat* se compare più volte nel testo e non è prolungabile né a destra né a sinistra mantenendo le occorrenze identiche. In un Suffix Tree, la massimalità a destra è intrinsecamente garantita dai nodi interni (che possiedono sempre almeno due figli, biforcandosi su caratteri diversi). Per garantire la massimalità a sinistra, è necessario che il nodo interno sia *left-diverse*. Le stringhe maximal repeat si trovano individuando tutti i nodi interni i cui sottoalberi contengono foglie associate ad almeno due caratteri precedenti ($T[p - 1]$) distinti.

6.2 Ricerca delle Supermaximal Repeat in T

Una *supermaximal repeat* è una maximal repeat che non è sottostringa di nessun'altra maximal repeat. Nel Suffix Tree, queste si individuano cercando nodi interni che soddisfano due condizioni tassative:

1. Tutti i loro figli devono essere esclusivamente nodi foglia (nessun figlio può essere un nodo interno).
2. I caratteri precedenti associati a tali foglie figlie ($T[p - 1]$) devono essere tutti distinti tra loro.

6.3 Estrazione delle posizioni in tempo lineare $O(n)$

Sì, è possibile fornire tutte le posizioni delle supermaximal repeat in tempo lineare $O(n)$, dove n è la lunghezza del testo T . La procedura è la seguente:

- Si costruisce il Suffix Tree in tempo $O(n)$.
- Si effettua una singola visita in profondità (DFS, in post-order) dell'intero albero, operazione che richiede tempo $O(n)$.
- Durante la visita, per ogni nodo, si verifica in tempo costante (o al più lineare rispetto al numero dei figli, con somma totale $O(n)$ su tutto l'albero) se rispetta le proprietà delle supermaximal repeat (solo figli foglia e caratteri di estensione a sinistra distinti).
- Una volta identificato un nodo supermaximale, le posizioni in cui la stringa occorre nel testo vengono estratte leggendo direttamente gli indici numerici registrati nelle sue foglie figlie. Poiché l'albero ha dimensione lineare e ogni foglia viene letta al più una volta, l'intero algoritmo garantisce un costo computazionale di $O(n)$.